AHA - A Decentralized Drone Delivery System

Arsh Jain, Arush Adabala, Henry Marks
Rice University
Houston, Texas, USA
{aj76, ara11, hem6}@rice.edu

***Abstract*—Drone delivery is a growing industry but has a glaring problem; Less than 1% of American businesses are serviced by drone delivery. This is resulting in many key businesses being neglected from the benefits that drone delivery provides. Currently, the problem prohibiting businesses from getting these benefits is the lack of co-location with droneports. All of the current models of drone delivery require that the drone ports be directly next to the businesses they service. We created the world's first decentralized drone delivery network to address this problem. A customer will place an order for an item on our website. One of the drone ports is then chosen to service this order. We dispatch a drone first to the business where the business gives the goods to the drone. The drone then files to a customer's home, where it finally fulfills the order. To provide this service we built 3 web interfaces: an ordering frontend, a frontend for the businesses to see orders come in, and a frontend for the droneport operators. Finally, this is all tied together with a simple database and a backend algorithm that manages the logistics, from choosing which drone to use to planning flight routes. With this infrastructure we're able to process up to 30 orders concurrently, only being limited by the amount of drones we have in service.**

I. Introduction

Drone delivery is the future we were promised, but for too long has been out of reach. And it is not for a lack of trying. From Amazon, to Flextrex, to Wing the space is actually quite competitive. In our modern world, drone delivery would solve many of consumers' most pressing challenges. We now expect food delivery in less than 30 minutes in less than 30 minutes to satisfy our craving for instant convenience, but current delivery systems are straining to keep up. Similarly, there are things that need to be shipped super securely, like jewelry, which would benefit from drone delivery. The current model of drone delivery is to have droneports directly adjacent to the businesses. Unfortunately, this is the bottleneck that is limiting the wide scale adoption of drone delivery technology. These drones can only support less than 1% of American businesses that can be serviced because it requires the colocation of the businesses to the droneport. This model is akin to the old versions of restaurant delivery models. Each restaurant would hire delivery drivers to service just their business. This model was disrupted by Doordash which made it possible for delivery drivers to service multiple different businesses. We plan on being the doordash to the drone industry.

II. Related Work

The body of research around drone delivery is large and rapidly growing. We base this Related Work section on a literature review of over 307 peer reviewed articles on the topic: "Drones in last-mile delivery: a systematic literature review from a logistics management perspective."

As background to the drone delivery industry we must define some key terms. According to the literature review, most of the literature generally agrees that the final person who the good is being sent to is known as the receiver. On the other hand, the sender is the individual who decides on which type of transportation to use. Some other key terms defined in the review that we use in our paper are customer/consumer. Scholars agree that these refer to the person who bought the good being sent, often but not always the receiver. In some cases however, drone delivery is used for other purposes beyond retail, so in this case consumer/customer would not apply. The literature review discusses 9 general topics of discussion emerging within the literature. For the purposes of this paper we examine the most important 3: Costs, Reach and Time. The review weighs the general trends in terms of the potentials and challenges discussed across all of the 307 papers. In terms of the costs, some of the potentials cited in the literature review were 23-93% potential cost reduction savings from drones and low investment and operating costs. While some of the challenges that exist are large upfront investments in drone fleets and infrastructure and all of the economic costs associated with maintaining the infrastructure.

As for the reach, the key benefit is that drones can access their destination without obstruction from obstacles like traffic. The challenge is the difficulty of finding good landing places in urban areas and inability to service remote communities. Finally in terms of the time factor the paper cites 60-79% reductions in delivery time, but at the same time the fact that shorter delivery times require using individual drones for each individual delivery. While the opportunities to save huge amounts of time and money saved by drones cited in the literature review are clearly large upsides, we believe that the downsides are addressable. In terms of the upfront and operational costs, we believe that much of the labor costs of operating droneports could be automated. This would help increase the margins and economic viability of the business model. We also believe that we can use artificial intelligence and data science to find optimal landing sites nearby to the customers and the businesses that the drones are picking up from. This would also help us address the urban landing site issue as we would be able identify more viable landing sites. Overall, we believe that the existing literature provides a solid economic and value case for drone delivery. We also believe that the challenges presented in the existing literature

are addressable by new technological approaches to these problems.

III. Methodology

We will now outline the system methodology and end-to-end data flow of our drone delivery and route planning platform. At a high level, the system exposes a set of HTTP APIs from a Go backend and an interactor with a relational database. database, mapping (e.g., Google Maps), PIs, and a web-based frontend. The backend receives constant delivery requests and discretizes them. transforms the operating region into a grid, and calculates feasible paths that do not violate no-fly zones. These routes are then returned to the frontend, displayed, and executed as NFZs.

1.1 System Overview

1.1 System Overview The design is based on an archetypal client-server approach. The browser-based front-end retrieves user inputs (e.g., depot locations, customer destinations, and NFZ settings) and transfers them to the Go backend through JSON via HTTPS. The backend uses this information to continue building a grid model of the area and subsequently appeals to a routing engine to calculate safe drone routes. The resulting paths are represented by sequences of latitude/longitude waypoints, which are serialized and sent back to the frontend to be displayed in an interactive map. downstream simulation. The Go backend is designed to be stateless with regard to long-running computations. Each HTTP request is considered a separate job and identified using an object identifier (UID) on a global scale. Extensive calculations (e.g., full-region grid generation) are divided into smaller tasks and are performed simultaneously with the help of goroutines and synchronization primitives. This will enable the system to scale with the number of CPU cores while maintaining the responsiveness of each request.

1.2 Data Model and Input Representation

1.2 Data Model and Input Representation The system contains a small, structured model of all the domain entities:

- Drone Ports / Depots: Each depot has a unique ID, internal operational parameters, lat, lng, maximum drones, turnaround duration, and service radius.
- Customer Requests: The delivery requests include a unique ID, origin, destination location, and coordinates, a requested start time, and metadata such as the package weight or delivery priority.
- No-Fly Zones (NFZs): NFZs are depicted as geometric shapes, typically circles or polygons with center points and radii. Each NFZ is also assigned a distinct ID and a type (e.g., permanent, temporary, weather-related).
- Grid Cells: The continuous area is approximated in grid cells, all of which are defined by their row / column index and their latitude/longitude boundary box. Each cell contains flags indicating whether it is blocked by an NFZ and information in the form of cache, such as approximate traversal cost.

These objects are all sent in the form of a JSON as communication between the frontend and backend, and are stored in a relational database so that they can be reproducible.

1.3 Discretization and Construction of Grids and Regions

1.3 Discretization and Construction of Grids and Regions. The fundamental aspect of the methodology is the grid-based discretization of the operating area. Instead of computing routes using continuous geometry, an overlay, which is a rectangular grid whose resolution is configurable, is used. The level of refinement is determined by a trade-off between route fidelity and computational cost. Grid building follows the following three steps:

- Bounding Box Determination: Using all depots, customers, and NFZs coordinates, we calculate a small bounding box for the back-end with a slight margin. The following bounding box is used to define the global grid extent.
- Cell Generation: The bounding box is further divided into equally spaced cells. We calculate the center coordinate of each cell and its spatial polygon.
- NFZ Marking: To state the safety constraints, the cell is examined to determine whether it has an intersection with each NFZ. In the case of circular NFZs, the distance between them is calculated as the area from the cell center to the NFZ center and highlights the cell as blocked in this case the distance is less than the NFZ radius. In the case of polygonal NFZs, the point-in-polygon test is used. Cells that intersect any NFZ are not computed for routes and are never examined.

This step results in a graph where each cell forms a node, and an edge connects them. The graph captures geographic limitations as well as regulatory limitations (NFZs) in a single, discrete structure.

1.4 Route Planning Algorithm

1.4 Route Planning Algorithm In order to compute a drone route, the backend snaps the target and depot locations on to the free grid cells as near as possible. We use the weighted shortest path algorithm, in which each edge between neighboring cells has a traversal cost. Costs can be defined as the physical distance between cell centres, or by other means of augmenting penalties, such as: the closeness to the boundaries of the NFZ, promoting greater safety margins, external map terrain, or infrastructure created based on map data, penalties of time indicative of the current winds or traffic. The routing process is given below:

1) Graph Construction: For the relevant subregion, construct an adjacency structure of non-blocked cells.
2) 2. Cost Initialization: Initialize distances to infinity except the source cell, which is set to zero.
3) 3. Search: Run the shortest-path algorithm until the target cell is reached or all reachable cells are exhausted.
4) 4. Path Reconstruction: Backtrack from the target cell using predecessor pointers to reconstruct the optimal path as a sequence of cells.

5) 5. Geometric Path Output: Convert the cell sequence into a polyline of latitude/longitude waypoints. Optionally, the backend may post-process the polyline to smooth turns and reduce the waypoint count.

The resulting polyline is further sent to the client as part of the API. response. Since the grid precalculates cell blocks that are blocked by NFZ, any path generated by this algorithm avoids flying over a no-fly zone.

1.5 External API Integration

1.5 External API Integration The backend contains integration with third-party services, mainly mapping and topography for road networks or elevation. The critical information is:

- Geocoding and Distance Estimation: The backend maps geocodes the raw coordinates into addresses, which can be read by humans. It may also query a distance matrix or routes API to compare the air-style estimates with road estimates based on distances.
- Generation of Map Overlays: NFZs, grid, and calculated routes are rendered into formats that can be displayed in the frontend mapping library (GeoJSON, or encoded polylines).

These integrations are wrapped in small client wrappers so they are not visible to the outside world. They are the only ones interacted with by the rest of the codebase through abstract interfaces. This abstraction simplifies testing and subsequent updates.

1.6 Parallelism and Job Management

1.6 Parallelism and Job Management. A back-end alternative for managing numerous simultaneous route calculations and grid changes is used and heavily uses Go's concurrency primitives. Each incoming HTTP request is also run in another goroutine. Prolonged activities, e.g., route calculations to make many deliveries or to regenerate the grid at a higher resolution, are decomposed into smaller jobs. These tasks are posted to workers' goroutines. The job-management approach will be as follows:

- Job Creation: In every logical work unit (e.g., compute route from A) to B), the server develops a job structure with the input parameters and a newly generated UUID.
- Queueing: The jobs are scheduled into in-memory queues guarded by a mutex or channels, and the en and de-queuing are made thread-safe.
- Worker Execution: There is a fixed number of worker goroutines that are listening to the job queues, pop the jobs, and make the respective routing or grid operations.
- Result Propagation: When a worker completes a job, it writes the result (or error) back into a result channel or shared map keyed by the job UUID. The HTTP handler either waits synchronously for the result or responds with the job ID so that the client can poll for completion.

The system uses multicore execution. It also simplifies the process, making it easier to impose limits on parallelism (e.g., maximum number of parallel routing jobs) and to apply high-load backpressure.

1.7 Simulation and Visualization Workflow

1.7 Simulation and Visualization Workflow The workflow of simulation and visualization involves three phases: planning, coding, and testing. After the computation of routes, the front end of the routes simulates and visualizes drone movements. The procedure of this layer is:

- Data Fetching: The frontend requests the backend for available depots, NAFZs, grid information, and computed paths. Responses are cached in the application state.
- Map Rendering: The frontend renders the overlay using a web mapping library, the grid shape, NFZ shapes, and flight paths. top of a base map. It is possible to color-block cells for highlighting constrained regions.
- Temporal Simulation: The frontend or backend calculates timestamps. at every waypoint of a route, with a model of drone speed. The frontend uses the drone's interpolated positions to animate the moving drones. The polyline gives users the chance to see the congestion, possible conflicts, and so on.

A. User Experiences

To illustrate the end-to-end flow, we include annotated screenshots of each UI:

- **Consumer (ordering)**: browse restaurants, place orders, see live drone markers, ETA, and delivery route polylines (merchant → customer).
- **Business (merchant)**: view incoming orders, confirm "loaded on drone" to release leg 2, see pickup routes and status up to handoff.
- **Droneport (ops)**: monitor the active order queue, live drone positions, leg 1/leg 2/return polylines, and per-port fleet cards (ready/in-flight/maintenance with battery).

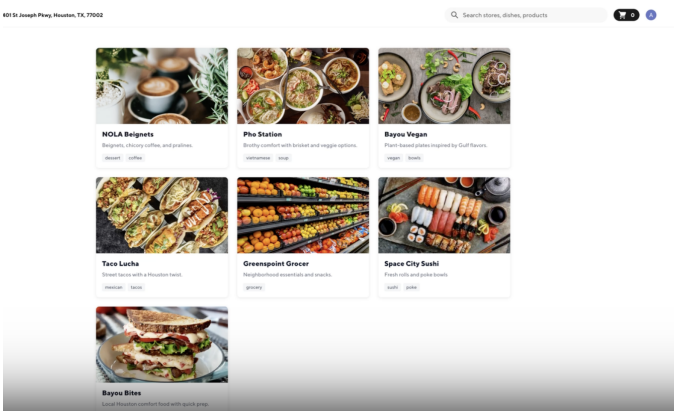

Fig. 1. UI for picking which restaurant to order food from.

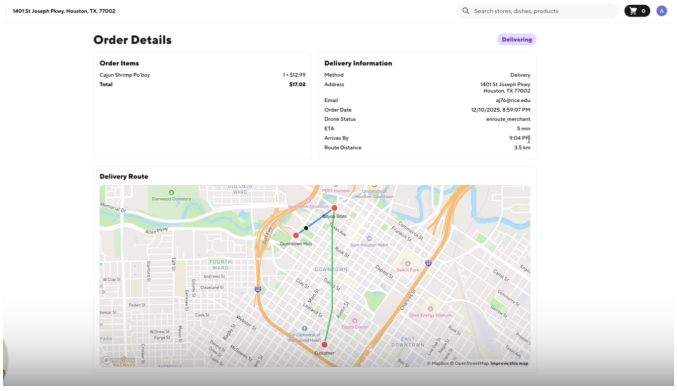

Fig. 2. Consumer experience with live route and ETA.

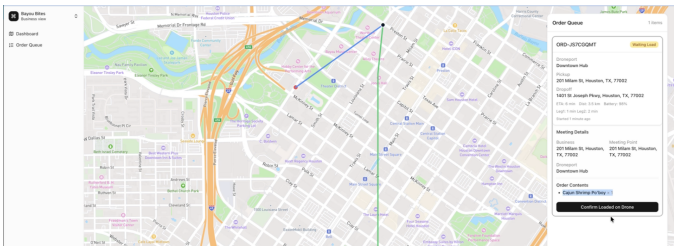

Fig. 3. Business dashboard showing order intake and load confirmation.

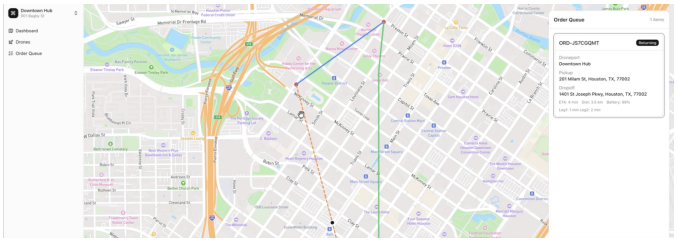

Fig. 4. Droneport dashboard with live routes, return leg, and fleet status.

B. End-to-End Demo Workflow

To illustrate the full lifecycle of an order in our system, we provide a recorded end-to-end demo that walks through the consumer, merchant, and droneport interfaces, along with the real-time routing and fleet updates performed by the backend. The demo can be viewed at:
here

The video shows the sequence from a customer placing an order, to merchant load confirmation, to drone dispatch, live route visualization, battery consumption, ETA updates, and the automated return-to-hub leg. This demonstrates the entire orchestration loop operating in real conditions.

IV. Evaluation

Evaluation

We evaluated our system using a simulation environment that generates and processes multiple concurrent delivery orders. The simulator seeds orders from different depots to diverse destinations all at once with varying request timings and spatial distributions. This puts pressure on both the routing engine and the concurrency mechanisms. Each order is routed over the grid, checked for conflicts with no-fly zones, and tracked from creation to completion. Successful runs are characterized by all orders being assigned, NFZ-compliant, processed without deadlocks or race conditions, and completed within acceptable latency bounds.

V. Conclusion

Conclusion

In conclusion, our group provided a model for drone delivery that, if implemented, could genuinely reshape the last mile delivery landscape. We demonstrated that the system can manage over 30 concurrent orders while routing drones in a highly optimal and efficient way, showing not only strong theoretical performance but also practical viability. Alongside the routing engine, we built full end to end infrastructure capable of taking in orders, assigning drones, and completing deliveries, which means that with actual hardware this could function as a real business rather than just a technical demo. Looking ahead, we plan to open source our orchestration model so others can build on it, refine it, or adapt it to their own use cases. And although we may not continue developing the project ourselves, we believe the routing techniques and system design we introduced represent meaningful and even groundbreaking steps forward and provide a strong foundation for future innovation in autonomous delivery systems.

Contributors and Contributions

A. Arush

My individual contributions were:

- I made the routing algorithms that operate on the grid. This included snapping coordinates to the grid, making the underlying adjacency graph, and running shortest-path algorithms. I tuned parameters like neighborhood connectivity and cost penalties so that the resulting routes balance safety, path length, and efficiency.
- I gathered and integrated all the external APIs, including mapping, geocoding, and related services. This involved selecting providers and understanding their data formats, and wrapping them in Go libraries. By exposing these services with interfaces, I made it so that the rest of the system could use geocoding and map overlays.
- I added the concurrency for routing jobs and fleet management. This included making job structures, setting up goroutines, and using channels and mutexes to enqueue, process, and collect results from multiple routing requests in parallel. I also prevented race conditions and deadlocks by carefully structuring sections and ensuring that shared states are accessed safely.
- I made the functions that dispatch drones to complete orders and maintain fleet information. This determines which drone to assign to a given order, updates drone status and enforces constraints like capacity and turnaround

times. The dispatch function is integrated with the routing layer so each drone gets a valid route.
- I designed the "ready" and "maintenance" buckets. The ready bucket tracks drones that are available for new orders. The maintenance bucket tracks drones that are unavailable. I implemented the state transitions between these buckets based on health.
- I implemented the functions that convert computed routes into polylines and sent them to the frontend. These functions turn paths into a series of latitude/longitude points and put the result in formats readable by the mapping library. This is good for debugging the routing algorithms and showing the end-to-end flow of the orders.

B. Henry Marks

- Built the 3 frontend UIs achieving clean UI and fast loading times
- Set up all of the data structures and the realtime backend convex server
- Set up and efficiently organized the project into a monorepo
- Built the authentication system using Clerk
- Lots of tailwind css and typescript lol

C. Arsh Jain

In the first stages of the project, I worked pretty closely with Arush in making sure that the backend was robust and reliable. Some things that I helped do in this stage include: fixing race conditions where multiple workers could take the same job by using locks, implemented battery tracking along the flight path, including predicting whether the drone can reach its target. I also updated the logging system (before we switched to convex) to better handle Which drones handled which deliveries, and whether each delivery was completed. Also for multi-leg routes, I added extra checks, so drones do not run out of battery between legs.

After Henry setup the Convex monorepo setup, I wired the Go router into Convex so it pulls orders and droneports from Convex, and the go code reports routing preogress with real polylines and durations (including load confirmation, the customer leg, and the return leg), sends fleet snapshots, and enforces maintenance. On the UI side, I added polyline rendering (taking the polylines from the backend and displaying them on the UI that Henry designed), live markers for the drone locations, and I also added the return to droneport logic as well as the UI polylines to show it on the droneport UI. I also made the batteries drain over time, leg3 timing, live ETAs, and fixed many visual bugs (e.g clear routes when nothing is selected; the business view hides orders once they leave the merchant). I also rebuilt the consumer order page to refresh every second, avoid map load errors, show the correct delivered status, and removed unused filters and battery display. I also created scripts that seed mock data for the simulation (starts up a convex mutation that adds 30 orders to the orders database), and another script to essentially reset the drones to all be at their respective hubs at 100% battery and set a certain amount of drones to each droneport available. These were very useful for testing. In short, I basically brought life to the frontend using the backend scripts. I recorded the demo shown in our presentation (the one of 30 concurrent orders), and I recorded the demo showcasing the full workflow of our product that I am showing here. I also wrote the README, and refactored the go code.

For AI use, for the go code in the beginning, I used it as a tool to look for bugs when I was confused (find race conditions). I also used it to write boilerplate code (i.e http requests, etc.). For the convex monorepo stuff, I used AI pretty heavily. I analyzed the frontend, and then I wrote a long 2 page prompt concerning the design choices and features that I wanted when I integrated the go backend into the 3 frontends. I then iterated back and forth, using claude code and going in manually to fix bugs as they occurred (sometimes random typescript errors would occur that prevented website loading). The mock data was completely AI generated since I didn't want to do that manually.

REFERENCES

[1] A. Jazairy, E. Persson, M. Brho, R. von Haartman, and P. Hilletofth, "Drones in last-mile delivery: a systematic literature review from a logistics management perspective," *International Journal of Logistics Management*, 2024.